7 The building blocks of policy

This chapter covers

- The structure of a Cedar policy: effect, scope, and conditions
- The PARC model (Principal, Action, Resource, Context) for representing access requests
- How Cedar evaluates access decisions using matching, condition logic, and deny-overrides
- Key features of the Cedar language, including strong typing, entity references, and namespaces
- Cedar's design principles that enable safety, performance, and formal analysis

At ACME Corp, the identity team has just finished rolling out a new set of access policies. Engineers can now deploy code only from company-managed devices, and only when they are on call or during business hours. Document sharing is managed through a graph of relationships, letting managers like Carol review their team's work without needing explicit access permissions. These decisions are made in real time, but behind the scenes, there must be rules that govern access.

This chapter is about those rules. After exploring the main models for dynamic authorization, we now focus on how policy is actually written and evaluated. What does it mean to say "on call" or "manager" in a way that a system can understand? How does an authorization engine decide whether to allow or deny a request?

To answer those questions, we'll use Cedar (https://www.cedarpolicy.com), a modern, open-source policy language designed for fine-grained, real-time access decisions. Cedar offers a clear and expressive way to define policies involving relationships, attributes, and contextual conditions, whether we're implementing ReBAC, ABAC, or PBAC.